架构表达：从需求到可讨论的结构模型

从“需求规格”进入“可评审的系统结构”

本讲问题

如何把脑子里的系统结构，变成团队可以讨论和评审的东西?

本讲产出

C1/C2/C3 架构图、核心场景时序图、图选择判断规则

课程连接

承接需求规格与质量属性场景，进入后续架构方案评审

课堂问题：如何把脑子里的系统结构，变成团队可以讨论和评审的东西?

概念框架

本讲位置：需求之后，架构决策之前

需求说明系统要解决什么问题，架构表达说明系统准备如何组织起来。

需求工程 → 架构表达 → 架构选型 → 测试交付

已完成
需求规格 V1、质量属性场景表、Persona 与核心场景

本讲新增
C4 分层视图、时序图、文本化图表工具

下一步
用这些模型进入 ADR、架构权衡和方案评审

M3 里程碑：模型包不是附录，而是评审入口

没有可讨论的结构模型，架构评审只能停留在口头争论。

M3

模型包 V1
C1 上下文图 + C2 容器图 + C3 组件图 + 1 张核心时序图

M4

架构方案 V1
说明为什么这样拆分、这样依赖、这样部署

M5

ADR 记录
记录关键选择、备选方案、权衡理由和后果

课堂动作 **课堂检查：自己的项目是否已经能画出 C1 和 C2？**

案例冲突

失败场景：三人分头开发，两周后无法合并

真正的问题不是代码写慢了，而是每个人对系统结构的假设不一样。

甲

从数据库出发建表，字段和前端期待的 JSON 不一致

→

乙

从前端出发写页面，假设后端接口已经按页面组织

→

丙

写一个大服务类，把认证、预约、审批逻辑耦合在一起

→

结果

接口、数据、职责边界在集成时集中爆雷

深层原因：没有共同结构模型，假设在集成时集中爆发。

易错点 课堂提问：这是沟通问题、接口问题，还是结构问题？

案例冲突

失败背后三个结构问题

架构表达要优先暴露边界、依赖和外部系统，而不是罗列功能。

边界模糊

哪些是系统内，哪些是外部服务，没有说清楚

依赖隐蔽

谁调用谁、谁依赖谁，在代码合并前没人看见

外部依赖缺失

认证、邮件、文件存储等外部系统没有进入讨论

架构表达是什么

架构表达是用约定清楚的图和文字，把关键结构决策外化出来。

架构表达是用约定清楚的图和文字，把关键结构决策外化出来。

不是

为了作业好看而画图，也不是 UML 图形大全

而是

让系统边界、职责划分、依赖关系和关键行为可见

结果

团队可以评审、质疑、修改，并把结论沉淀进版本库

考试锚点 **名词解释：架构表达。**

为什么选择 C4：先统一图的层级

很多架构图失效，不是因为画得少，而是因为没有说明自己在讲哪一层。

普通图的问题

同一张图里混合用户、系统、模块、数据库、类和接口

vs

C4 的做法

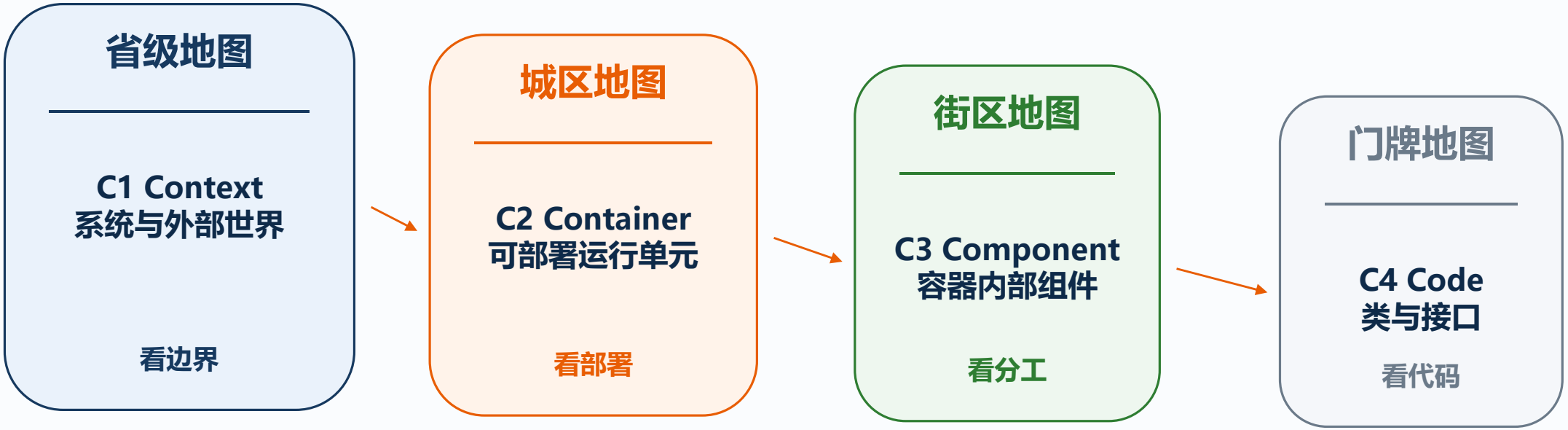
强制区分上下文、容器、组件、代码四个抽象层级

课堂收益：学生先学会判断层级，再学习具体图形符号

概念框架

C4 像地图缩放：每一层回答不同问题

不要用一张图回答所有问题，应该用不同层级回答不同问题。



同一系统需要多张“比例尺不同”的图，不能用一张图塞进所有层级。

C4 四层视图总览

C4 的核心不是四张固定图，而是四个逐步深入的抽象层级。

C1 Context 系统边界与外部世界

C2 Container **可独立部署运行单元**

C3 Component 容器内部组件分工

C4 Code 类、接口、代码结构

记忆动作：先问这张图回答哪一层的问题，再判断图中的元素是否同层。

方法拆解

C1 上下文图：先画系统边界和外部世界

C1 不关心系统内部怎么做，只回答谁使用系统、系统依赖谁。

C1 不关心系统内部怎么做，只回答谁使用系统、系统依赖谁。

中心

被讨论的系统本身

外部角色

学生、教师、实验员、管理员等使用者

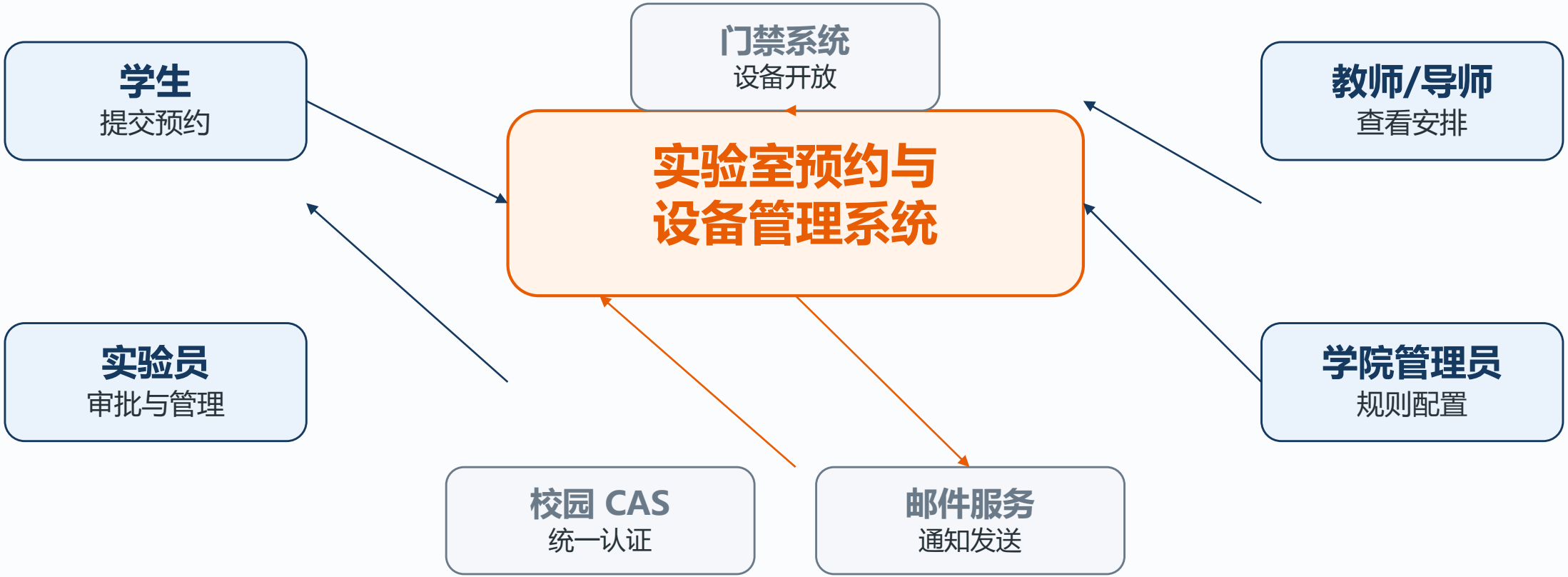
外部系统

校园认证、邮件服务、门禁系统、学院 HR 等

考试锚点 简答题：C1 图回答什么问题？

实验室预约系统 C1 示例

先把系统放在校园生态里，才能看见真正的边界。



练习/考试

C1 常见错误：把世界画进系统里

C1 的边界判断标准是：这个东西是否由我们开发、部署和维护。

错误一

把校园认证画成自己系统的一个模块

错误二

只画学生，不画审批人、管理员和运维方

错误三

把数据库画到 C1 里，提前进入内部实现

课堂动作 判断题：MySQL 数据库是否应该出现在 C1 图里?

C2 容器图：本讲最关键的一张图

C2 的“容器”不是 Docker 容器，而是可独立部署的运行单元。

C2 的“容器”不是 Docker 容器，而是可独立部署的运行单元。

可以是

Web 前端应用、后端 API 服务、数据库、消息队列、缓存

不是

用户管理模块、预约管理模块、审批函数、某个 Java 类

判断标准

能否独立部署、独立启动、独立扩缩容、拥有清晰运行边界

考试锚点 **名词解释：C2 容器中“容器”的含义。**

练习/考试

判断一个框是不是 C2：问三个问题

画 C2 前先判断运行边界，不要把功能模块误画成容器。

能独立部署吗

是否可以单独发布和回滚

能独立启停吗

是否有自己的运行进程或服务实例

能独立扩缩容吗

是否可以按负载单独扩展容量

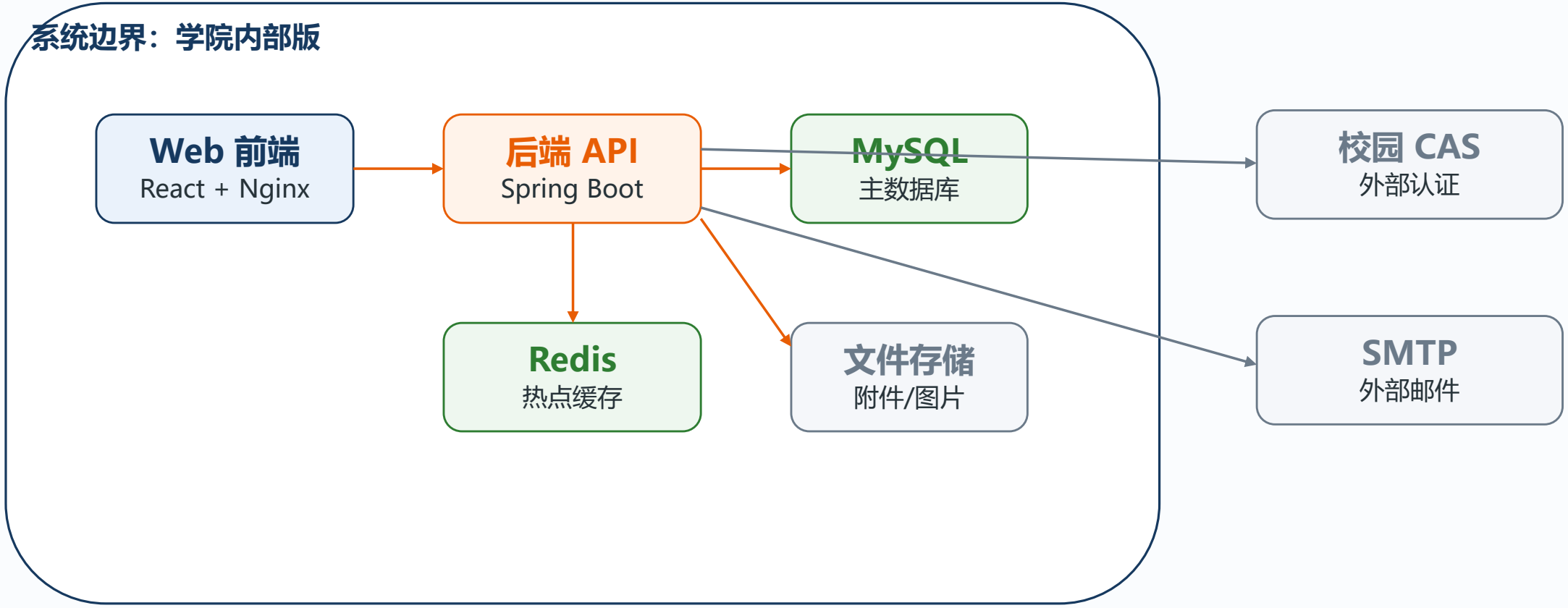
三问都否

大概率是 C3 组件或代码内部模块

课堂动作 课堂投票：用户管理模块、消息队列、数据库分别是不是 C2？

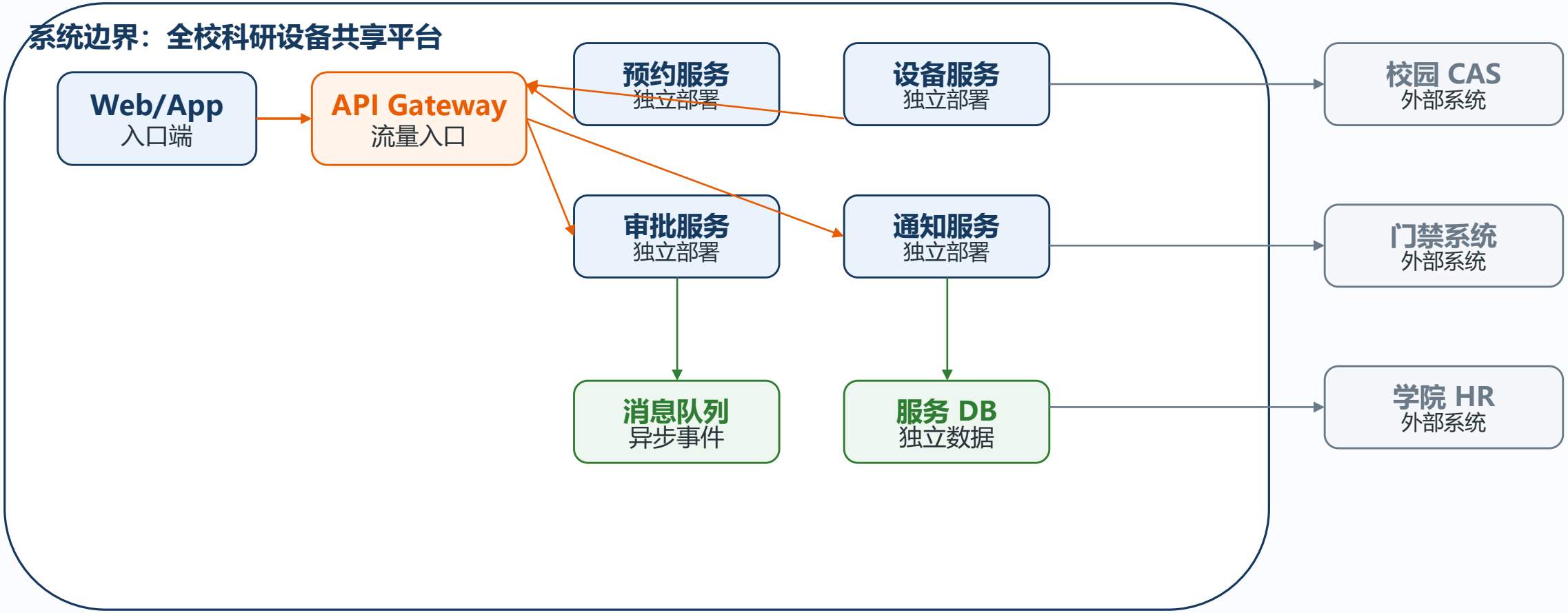
学院内部版 C2：模块化单体足够解决问题

并发规模小、团队小、维护成本敏感时，简单结构往往更合理。



全校平台版 C2：服务边界开始变成架构问题

当并发、可用性、组织协作和扩展性提升时，容器划分会随之变化。



案例冲突

同样的预约功能，为什么 C2 完全不同

功能需求相似，不代表架构结构相同；质量属性才是关键分水岭。

学院内部版

低并发、团队小、维护交接敏感。 结构：Web + API + MySQL + Redis。

全校平台版

高并发、跨学院、审计和可用性要求高。 结构：Gateway + 多服务 + MQ + 独立数据。

并发	100 → 1000+
可用	工作时段 → 7×24
治理	单学院 → 跨学院
扩展	低频 → 持续演进

C2 质量标准一：边界清晰

每个容器框都必须代表一个可独立运行和部署的单元。

合格

Web 前端、API 服务、数据库、消息队列

不合格

用户管理、预约管理、审批管理等功能模块

追问

这个框能不能单独发布？能不能单独出故障？

C2 质量标准二：依赖方向合理

箭头不是装饰，箭头方向代表谁调用谁、谁依赖谁。

把每条箭头读成一句话：A 调用 B 完成什么？读不通就要改。

C2 质量标准三：外部依赖不遗漏

C1 里出现的外部系统，进入 C2 时不能凭空消失。

C1 中的外部系统	进入 C2 后是否保留	遗漏后果
校园 CAS	后端 API → CAS	认证与权限设计缺失
邮件服务	通知服务 / API → SMTP	审批结果无法通知
门禁系统	预约通过 → 门禁开放	业务闭环断裂

考试锚点 **简答题：为什么 C2 必须保留外部依赖?**

练习/考试

高频错误：把功能模块当容器

C2 看运行单元，C3 才看容器内部的功能组件。

错误画法

用户管理模块 → 预约管理模块 → 设备管理模块

为什么错

它们可能都在同一个 Spring Boot 进程里

纠正画法

C2 画后端 API；C3 再展开 API 内部模块

课堂动作 考试提醒：判断一个框属于 C2 还是 C3。

C3 组件图：进入一个容器内部

当 C2 已经说明运行边界后，C3 才用来说明容器内部怎么分工。

当 C2 已经说明运行边界后，C3 才用来说明容器内部怎么分工。

适用对象

某一个容器内部，例如后端 API 服务

典型组件

Controller、Service、Repository、Rule Engine、Notification Adapter

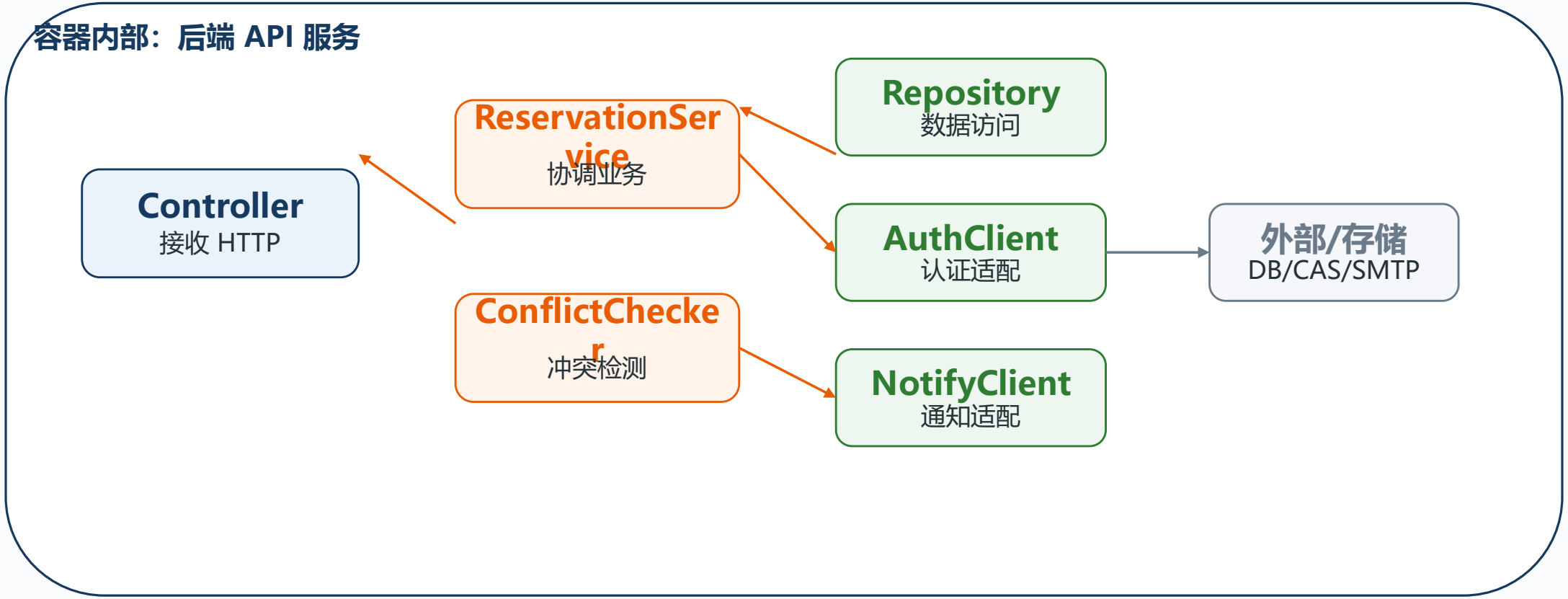
使用条件

内部职责复杂、多人并行开发、需要说明模块边界时

考试锚点 简答题：C2 与 C3 的区别。

后端 API 服务 C3 示例

C3 图要解释容器内部职责，而不是重复 C2 的部署结构。



C1/C2/C3/C4 速查表

判断一张图是否合格，先看它回答的问题是否和层级匹配。

层级	回答的问题	典型元素	课程项目要求
C1	系统和外部世界如何连接	用户、外部系统	必须有
C2	内部有哪些可部署单元	Web/API/DB/MQ	必须有
C3	容器内部如何分工	Controller/Service/Repo	重点容器有
C4	代码类和接口如何组织	类、接口、包	按需使用

概念框架

只有结构图还不够：还要看系统怎么动起来

C2 说明系统有什么，时序图说明一次业务请求如何穿过这些容器。

C2 静态结构

系统中有什么容器？ 谁依赖谁？ 外部系统在哪里？

时序图动态行为

一次请求怎样流动？ 哪里同步等待？ 异常路径如何处理？

结构 × 行为互验

时序图基本元素

时序图用参与者、生命线和消息，表达一次场景的时间顺序。

1

参与者

应优先对应 C2 容器，而不是模糊功能名

2

生命线

表示对象或容器在时间上的存在

3

消息

表示调用、返回、异步通知或自调用

4

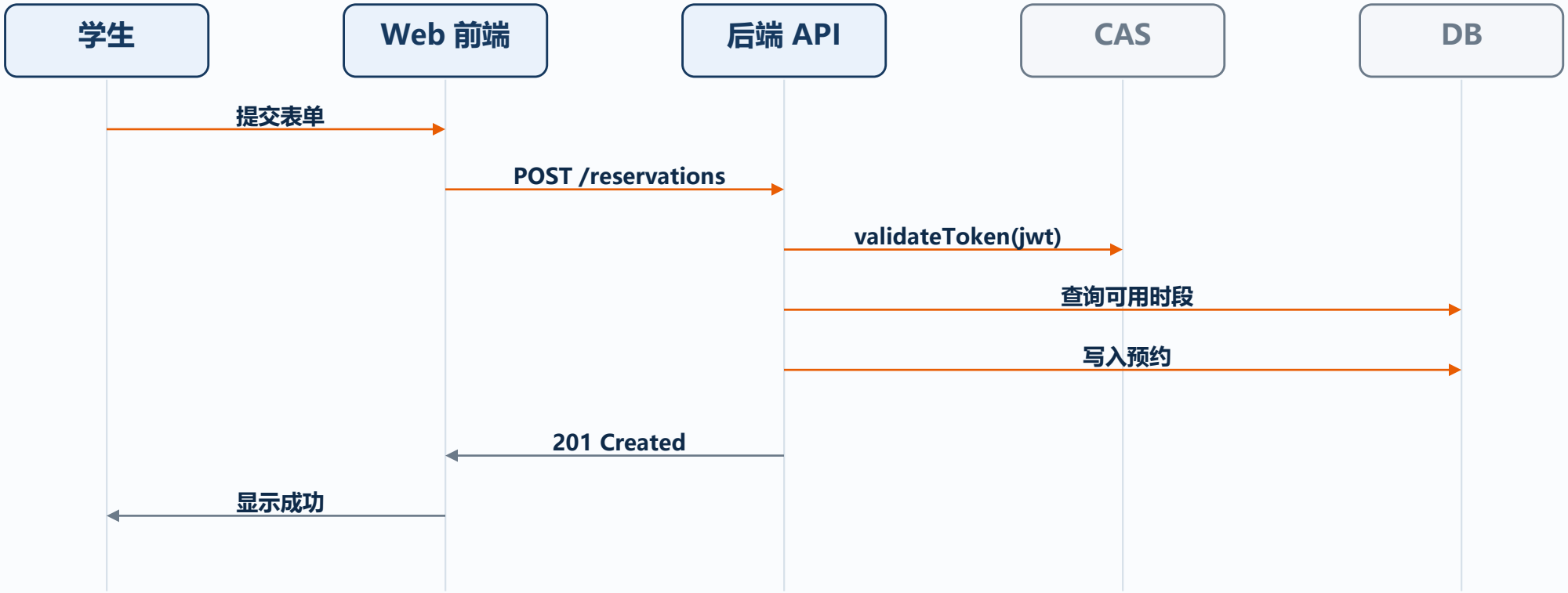
分支

用 alt/else 表达主路径和异常路径

方法拆解

预约提交主路径时序图

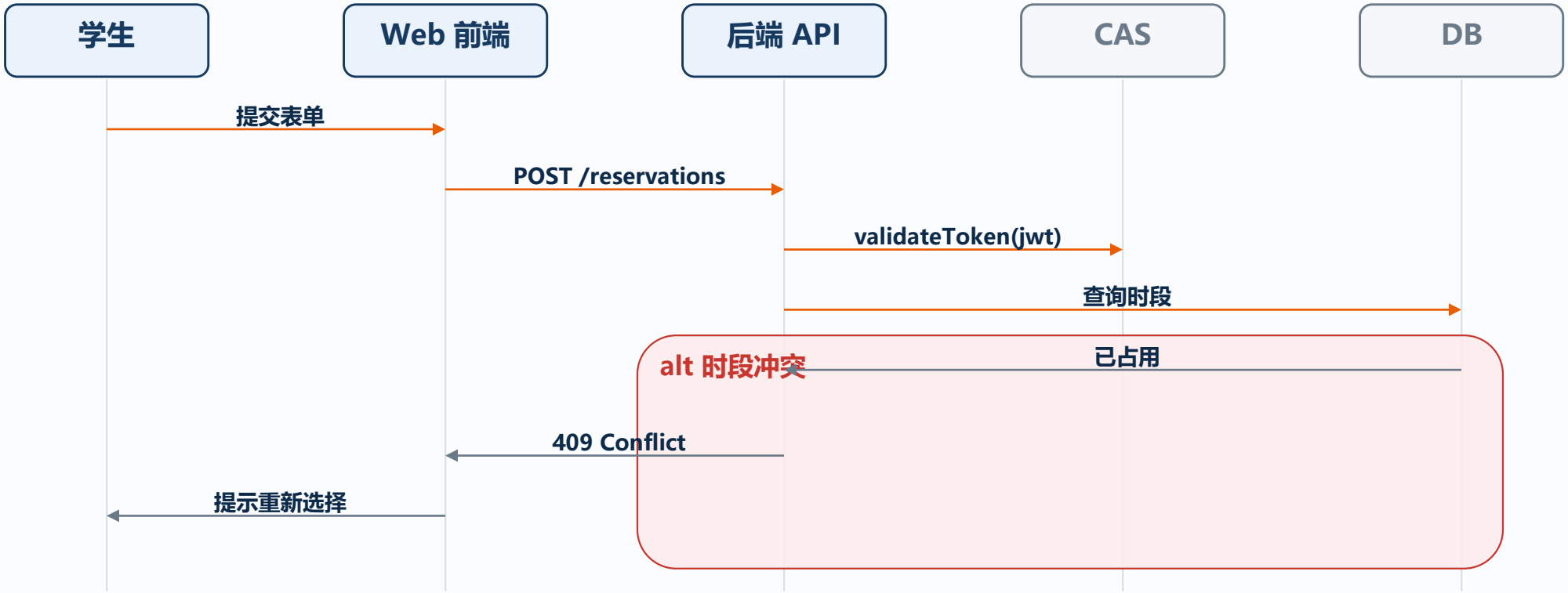
主路径要让人看清用户动作、接口调用、认证检查和数据写入。



方法拆解

加入异常路径：时段冲突

没有异常路径的时序图，往往只描述了理想世界。



时序图三条规范

一张好时序图要控制范围、命名清楚、包含异常。

参与者控制

只放与该场景直接相关的容器或外部系统

消息有意义

写 validateToken(jwt)，不要写“处理一下”

包含异常

至少有一条典型异常路径，例如 409 Conflict

关系结构

C2 与时序图互验：结构和行为互相找漏洞

时序图中出现的参与者，应该能在 C2 图里找到对应容器或外部系统。

从 C2 到时序

每个关键调用链是否能穿过现有容器？ 容器之间的依赖方向是否支持这个场景？

→ **互验** →

从时序到 C2

时序图里出现的参与者，是否都能在 C2 中找到？ 有没有漏掉缓存、队列或外部服务？

模型不一致就是设计风险：图之间必须能互相解释。

方法拆解

文本化架构图：让图进入工程协作

图如果只能存在于个人电脑上的截图，就很难成为团队资产。

1

可版本管理

Mermaid/Plant UML 源码可以放进 Git

2

可 Diff

图的变化可以像代码一样被审查

3

可协作

多人可以在 PR 中讨论图的修改

4

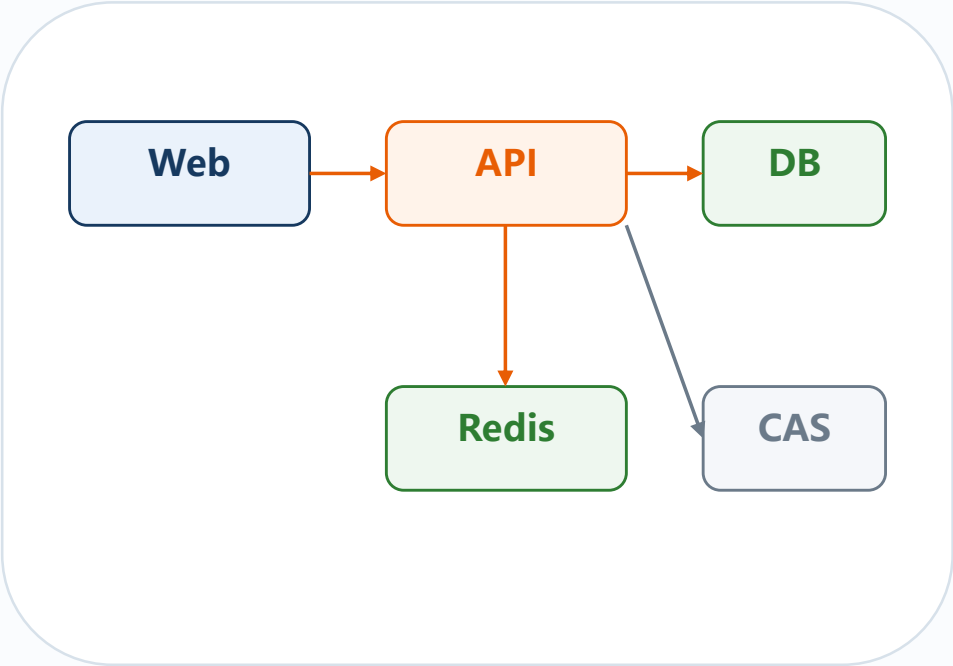
可自动化

CI 或文档系统可以自动渲染最新图

Mermaid 示例：用文本表达 C2 容器关系

现场演示的重点不是语法，而是文本和图形可以同步演进。

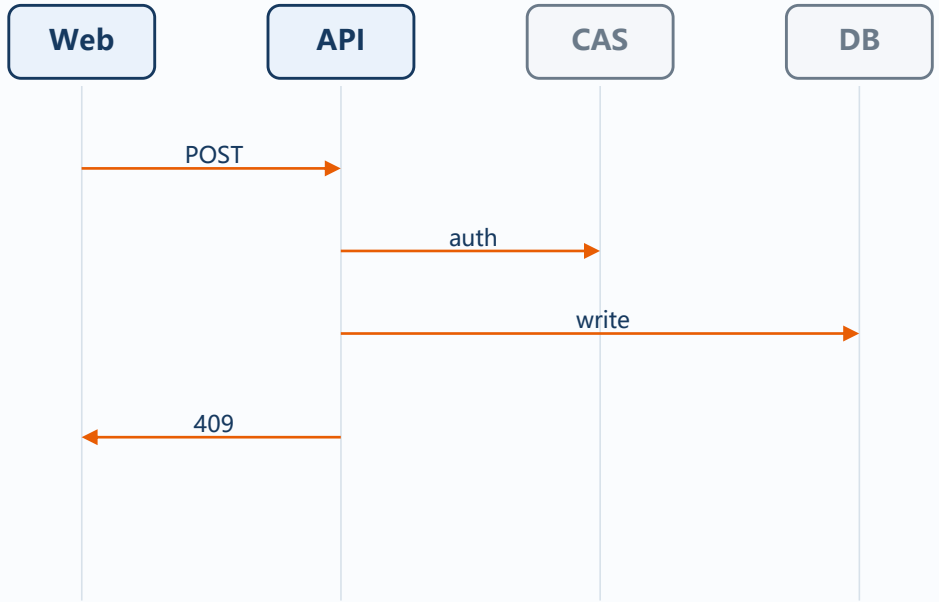
```
graph LR
  Web --> API
  API --> DB
  API --> CAS
  API --> SMTP
  API --> Redis
```



Mermaid 示例：用 sequenceDiagram 表达行为

文本化时序图适合和接口设计、评审记录一起维护。

```
sequenceDiagram
  Web->>API: POST /reservations
  API->>CAS: validateToken(jwt)
  API->>DB: insert reservation
  alt conflict
    API-->>Web: 409 Conflict
  end
```



工具选择：不是越复杂越专业

工具选择要服从协作方式、图的生命周期和团队熟悉度。

工具	适合场景	优点	注意点
Mermaid	Markdown / PR / 轻量图	可版本管理	复杂图表达有限
PlantUML	正式 UML / 文档库	表达能力强	语法学习成本
draw.io	快速讨论 / 白板	上手快	diff 较弱
Structurizr	C4 长期维护	模型驱动	配置成本更高

方法拆解

AI 可以帮画图，但不能替你做边界判断

AI 能生成草图，架构师必须判断层级、边界、依赖和质量属性是否合理。

AI 可以辅助

把文字需求整理成 C1/C2 草图 生成 Mermaid 初稿 检查是否遗漏角色和外部系统 把图说明改写成评审语言

AI 不能替代

判断容器边界是否真实可部署 判断质量属性权衡是否合理 决定哪些依赖可接受 承担架构决策后果

提示词原则：先让 AI 列假设，再让它画图，最后由人逐条审查边界和依赖。

方法拆解

UML 工具箱：知道什么时候用什么图

本讲不讲 UML 大全，只训练图的选择判断。

用例图

表达系统边界、参与者和用户目标

类图

表达核心领域对象、属性和关系

状态图

表达对象状态变化，例如预约单状态

部署图

表达运行节点、网络和部署关系

方法拆解

遇到什么问题，用什么图

图的选择取决于你要回答的问题，而不是工具菜单里有什么图。

遇到的问题	推荐图	理由
谁使用系统，系统边界在哪里	C1 / 用例图	表达角色与系统目标
系统内部如何部署和依赖	C2 / 部署图	表达运行单元与节点
核心对象如何关联	类图	表达领域对象结构
一次业务如何交互	时序图	表达消息顺序
对象状态如何变化	状态图	表达状态迁移

练习/考试

快速判断：这些问题该用什么图

能选对图，说明你理解了问题的抽象层级。

问题 A

学院领导想知道系统依赖哪些校级平台

C1 / 用例图

问题 B

开发组想知道后端内部哪些组件负责冲突检测

C3

问题 C

测试组想知道预约单从提交到取消的状态变化

状态图

问题 D

运维想知道哪些服务部署在哪些节点上

部署图

练习/考试

课堂演练：为课程项目画 C2 + 时序图

今天的图不是课后装饰，而是 M3 架构评审的直接输入。

1

任务 1
画 C2：系统内可独立部署单元 + 外部依赖 + 依赖方向

2

任务 2
画时序图：核心场景主路径 + 至少一条异常路径

3

任务 3
用三条标准互评，并记录需要修改的地方

课堂动作 **课堂产出：每组至少一张 C2 草图和一张时序草图。**

练习/考试

演练时间盒：先画结构，再画行为

不要一开始就纠结细节，先把边界和调用链画出来。

5 分钟

动作

列出使用者、外部系统和可部署单元

8 分钟

动作

画 C2 容器图并标注依赖方向

7 分钟

动作

选一个核心场景画时序图

5 分钟

动作

同组或邻组互评，指出两个具体问题

练习/考试

互评三条标准

评审时不要说“还可以”，要指出具体框、具体箭头、具体遗漏。

边界清晰

每个框是否能独立部署和启停?

方向合理

箭头是否清楚表达谁调用谁?

依赖完整

C1 中的外部系统是否在 C2 中保留?

课堂动作 **评分依据：综合案例题可按三条标准扣分。**

练习/考试

讲评：四类高频问题

图画错不可怕，关键是能用规则把错误定位出来。

功能当容器

用户管理/预约管理/设备管理应先判断是否独立部署

外部系统放错

CAS、邮件、门禁不要画进自有系统内部

消息命名模糊

用 POST /reservations，而不是“请求一下”

缺异常路径

只画成功路径，无法暴露并发冲突和失败反馈

课堂动作 考试提醒：案例题要说明为什么错、如何改。

本讲考试锚点

考试不会考画图软件语法，但会考你能不能选对图、画对边界。

名词解释

C4 四层定义；C2 容器含义

简答题

C2 与时序图分别回答什么问题

案例题

给需求和质量属性，画 C1/C2 草图并解释决策

判断题

给场景选择 C4、时序图、用例图、类图、状态图或部署图

三条必须带走的结论

能被评审的系统结构，才可能进入真正的架构设计。

1

结论一
架构表达让隐含假设变成可见结构

2

结论二
C2 是连接需求、部署和团队协作的关键层级

3

结论三
时序图用行为反查结构，能暴露 C2 漏洞

课后任务：完善 M3 模型包 V1

课后重点不是重新画漂亮图，而是把边界、依赖和核心流程修准确。

必交 1
C1 上下文图：使用者、外部系统、系统边界

必交 2
C2 容器图：可部署单元、依赖方向、外部依赖

必交 3
1 张核心场景时序图：主路径 + 至少 1 条异常路径

自查
用三条 C2 标准和三条时序图规范检查后再提交

问题交流